# Lecture 6: Instruction Set, Registers, Processor Speed and Types of Processors

**Instruction Set · Registers · Processor Speed · Intel, AMD, ARM · RISC vs CISC**

## Learning Outcomes

- Recall the CPU, ALU, Control Unit and instruction cycle from Lecture 5 and connect them to today's deeper look inside the CPU.
- Define an instruction and an instruction set, and explain why different CPUs can understand different instructions.
- Name and describe the function of the key CPU registers: PC, MAR, MDR, IR, and the Accumulator.
- Explain processor (clock) speed in GHz and describe how multi-core processors allow several tasks to run at once.
- Differentiate between clock speed and number of cores, and explain why a higher GHz does not always mean a faster computer.
- Compare Intel, AMD and ARM as processor designers/manufacturers, and identify typical devices that use each.
- Explain the RISC and CISC design philosophies, and identify which processor families follow each.
- Answer short, medium, and long-answer (up to 15 marks) examination questions on this topic with full explanations, diagrams, and examples.

# 1 Recap — From the CPU's Parts to How It Actually Runs

In Lecture 5, we opened up the CPU and met its three internal parts: the Control Unit, the Arithmetic Logic Unit (ALU), and Registers. We also met the instruction cycle — Fetch → Decode → Execute — repeated endlessly for every single instruction a program contains.

Today we go one level deeper into each of those ideas. We ask: what exactly is being fetched — an "instruction" from an "instruction set" — and what does that mean? What exactly do those registers hold, and why are there several different named ones? How fast does this cycle actually happen, and why do two computers with the "same" clock speed sometimes feel completely different? And finally — why isn't there just one kind of CPU?

# 2 The Instruction Cycle and the Instruction Set

A program is nothing but a list of instructions stored in memory, waiting for the CPU to fetch and carry out one at a time, in order (unless a jump/branch instruction changes that order). The Control Unit drives this endless three-step loop:


*Fig 6.1 — The instruction cycle, recapped from Lecture 5, Fig 5.3.*

## What Is an Instruction Set?

An **instruction** is a single basic command the CPU can carry out — such as "add these two numbers," "move this value from memory into a register," or "compare these two values." The **instruction set** (also called the **Instruction Set Architecture**, or ISA) of a CPU is the complete, fixed vocabulary of every instruction that model of CPU is built to understand and execute.

**Key point:** Different CPU families can have different instruction sets. This is one major reason why an app compiled for one type of chip (e.g. an ARM-based phone) will not run directly on a different type of chip (e.g. an Intel-based laptop) without translation or emulation.

**Connecting back to Lecture 1:** recall the layers of translation — your code is translated into assembly language, then into machine code (pure binary). That machine code is nothing but instructions from the CPU's own instruction set, written as binary patterns the Control Unit can fetch and decode.

| Example instruction | What it does |
|---|---|
| LOAD | Copy a value from memory into a register. |
| STORE | Copy a value from a register into memory. |
| ADD / SUB | Perform addition or subtraction using the ALU. |
| COMPARE | Check whether two values are equal, greater, or smaller (a logical operation). |

| Example instruction | What it does |
| --- | --- |

**Real-Life Example**

Two phones might both use ARM-family chips, so an app built for one can often run on the other. But a Windows program built for an Intel or AMD (x86-64) laptop cannot run directly on an ARM-based Chromebook, because the two chips understand different instruction sets — the words in one chip's "vocabulary" simply don't exist in the other's.

# 3 Registers — The CPU's Own Fast Memory, in Depth

Lecture 5 introduced registers as the CPU's own tiny, extremely fast storage, holding only the handful of values needed for the instruction being executed right now. In fact, the CPU has several different registers, each with its own specific job during the instruction cycle.

| Register | Full name | What it holds |
| --- | --- | --- |
| PC | Program Counter | The memory address of the NEXT instruction to be fetched. |
| MAR | Memory Address Register | The address of the memory location currently being read from or written to. |
| MDR | Memory Data Register (also called the Memory Buffer Register, MBR) | The actual data being transferred to or from memory. |
| IR | Instruction Register | The instruction currently being decoded and executed. |
| AC | Accumulator | The running result of the ALU's current arithmetic or logical operation. |
| GPRs | General Purpose Registers | Temporary working storage for values the CPU is currently processing. |

## How the Registers Work Together During One Fetch

The registers above are not independent — they hand data to one another in a fixed order during every single Fetch step:

1. The Program Counter (PC) holds the address of the next instruction.
2. That address is copied into the Memory Address Register (MAR).
3. The CPU reads memory at that address; the instruction found there is placed into the Memory Data Register (MDR), then copied into the Instruction Register (IR) for decoding.
4. The Program Counter (PC) is incremented, ready to point to the following instruction.

**Memory Trick for Exams**

"MAR before it goes (the address), MDR when it arrives (the data), PC always points forward, IR holds what's now."

# 4 Processor Speed — Clock Speed and Multi-Core

Every CPU contains an internal clock that "ticks" at an extremely steady, extremely fast rate. On every tick, the CPU can begin a new step of work. This rate is called the clock speed (or clock frequency), and is measured in Hertz (Hz). Modern CPUs run at billions of ticks per second, so their speed is given in GHz (gigahertz).

| Term | Meaning |
|---|---|
| 1 Hz | One tick of the clock per second. |
| 1 GHz | One billion (1,000,000,000) ticks per second. |
| A 3.5 GHz processor | Completes roughly 3.5 billion clock ticks every second. |

## Multiple Cores — Doing More Than One Thing at Once

A single processing core can only work on one instruction stream at a time. Modern CPUs pack multiple independent cores onto a single chip — dual-core (2), quad-core (4), octa-core (8), and beyond — allowing several programs, or several parts of the same program, to run truly simultaneously rather than one after another.

| Setup | 8 tasks arrive at once — what happens |
|---|---|
| 1 core | Tasks are handled one at a time, in a queue — the 8th task waits for the 7 before it to finish. |
| 4 cores | Up to 4 tasks can run at the exact same moment, so the whole batch of 8 finishes roughly 4 times faster. |

**Common Mistake**

Clock speed tells you how fast ONE core works through its own steps. Core count tells you how many separate streams of work can run at the same time. A higher GHz does not let a single-core-only task magically use multiple cores, and more cores do not speed up a task that can only ever run on one core at a time.

**Beyond GHz and cores:** real-world performance is also shaped by cache size (small, very fast memory built into the CPU, bigger than registers but smaller and faster than RAM), whether the processor is 32-bit or 64-bit (how much data it can handle in one go), and how efficient its instruction set design is — which brings us to RISC and CISC.

# 5 Types of Processors — Intel, AMD and ARM

Almost every processor you will ever encounter — in a laptop, desktop, tablet, or phone — comes from one of three major processor families: Intel, AMD, and ARM.

### Intel

The oldest and largest of the three, Intel designs and manufactures its own chips using the x86-64 instruction set. Intel processors are found in the majority of desktop and laptop computers — the Core i3/i5/i7/i9 range for consumers, and the Xeon range for servers.

### AMD

Intel's main rival, AMD also designs and manufactures x86-64 compatible chips — the Ryzen range for consumers and the EPYC range for servers — and often competes closely with Intel on price and performance, with software generally running on either brand without modification.

### ARM

Unlike Intel and AMD, ARM (originally Acorn RISC Machine, now Advanced RISC Machines) does not usually manufacture chips itself. Instead, ARM designs the ARM instruction set and processor architecture, and licenses it to other companies — such as Qualcomm (Snapdragon), Apple (the A-series and M-series chips), Samsung (Exynos), and MediaTek — who then build their own actual chips around it. ARM-based chips dominate smartphones and tablets, and are now increasingly used in laptops too, largely because of their power efficiency.

| Feature | Intel | AMD | ARM |
|---|---|---|---|
| Instruction set family | x86-64 (CISC) | x86-64 (CISC) | ARM (RISC-based) |
| Designs AND manufactures own chips? | Yes | Yes | Usually only designs; licenses the architecture to others |
| Typical devices | Desktops, laptops, servers | Desktops, laptops, servers | Smartphones, tablets, increasingly laptops |
| Known for | Broad market leadership; Core / Xeon lines | Strong price-to-performance; Ryzen / EPYC lines | Power efficiency and battery life |
| Example chips | Core i7, Xeon | Ryzen 7, EPYC | Snapdragon, Apple M-series, Exynos |

**Real-Life Example — Reading a Laptop Spec Sheet**

If a laptop's spec sheet says “Intel Core i5” or “AMD Ryzen 5,” it is an x86-64 machine, typically running Windows or Linux. If it says “Apple M-series” or “Snapdragon,” it is an ARM machine — Apple's MacBooks with M-series chips run macOS built for ARM, while Snapdragon-powered laptops typically run Windows for ARM.

| Feature | Intel | AMD | ARM |
|---|---|---|---|

## 6 RISC vs CISC — Two Philosophies of Chip Design

Beyond the company that makes a chip, processors also differ in a deeper way: the design philosophy behind their instruction set. This is the difference between RISC and CISC.

### RISC — Reduced Instruction Set Computer

RISC chips use a small number of simple instructions, each doing one basic job and usually completing in about a single clock cycle. Because each instruction is simple, the hardware itself can be simpler and more power-efficient — but a program written for a RISC chip generally needs more individual instructions to accomplish the same task, since complex operations must be built up from many small ones.

**Used by:** ARM-based chips (almost all smartphones), Apple's M-series chips.

### CISC — Complex Instruction Set Computer

CISC chips use a larger, richer set of instructions, where a single instruction can carry out several steps at once — for example, fetching a value from memory and performing a calculation on it in one instruction. This means a CISC program often needs fewer instructions overall, but the hardware needed to support these complex instructions is itself more complex.

**Used by:** Intel and AMD's x86-64 chips (most traditional laptops and desktops).

| Point | RISC | CISC |
|---|---|---|
| Instruction complexity | Small, simple, fixed-length instructions | Larger, complex, variable-length instructions |
| Instructions needed per program | More instructions needed | Fewer instructions needed |
| Time per instruction | Usually about 1 clock cycle | Can take multiple clock cycles |
| Hardware design | Simpler circuitry, more power-efficient | More complex circuitry |
| Compiler's role | Does more work breaking tasks into simple steps | Simpler compiling; the CPU does more per instruction |
| Used by | ARM, Apple M-series | Intel, AMD (x86-64) |
| Typical devices | Smartphones, tablets, some modern laptops | Traditional desktop and laptop computers |

| Point | RISC | CISC |
| --- | --- | --- |

### An Everyday Analogy — Lego Bricks vs a Multi-Tool

RISC is like building something out of simple, identical Lego bricks: no single brick does very much on its own, but you can snap together large numbers of them very quickly and predictably. CISC is like using a Swiss Army knife: it has fewer separate tools, but each one — the can-opener, the scissors, the screwdriver — can complete a more complex job in a single action.

## 7 Putting It All Together — Reading a Spec Sheet

Suppose you are comparing two laptops before buying one:

| | Laptop A | Laptop B |
| --- | --- | --- |
| Processor | Intel Core i5 | Apple M2 (in a MacBook Air) |
| Family / design | x86-64 — CISC | ARM — RISC |
| Clock speed | 2.4 GHz | 3.5 GHz |
| Cores | 4 cores | 8 cores |

Each part of that spec sheet is one of today's ideas:

- “Intel / AMD / ARM” tells you the processor family and hence the instruction set it uses — and broadly, whether it is CISC (Intel/AMD) or RISC (ARM).
- “GHz” tells you how fast a single core ticks through its own steps.
- “Cores” tells you how many independent tasks or threads can genuinely run at the same time.
- Together, these three specifications explain raw performance, battery life (RISC/ARM chips are generally more power-efficient), and software compatibility (x86-64 apps generally will not run on ARM without translation).

**Common Doubt — Does More GHz Always Mean a Faster Laptop?**

Not necessarily. In the table above, Laptop B has both a higher clock speed AND more cores than Laptop A, so it would usually feel faster overall — especially for multi-tasking. But if you only ever run one simple, single-threaded task, the number of cores matters far less, and the efficiency of each individual instruction (RISC vs CISC) plays a bigger role than the GHz number alone.

## 8 Fill in the Blanks

- 1. The complete, fixed vocabulary of commands a CPU understands is called its _____.
- 2. The register that holds the address of the NEXT instruction to be fetched is the _____.
- 3. The register that holds the data currently being transferred to or from memory is the _____ (also called the Memory Buffer Register).
- 4. The register that holds the instruction currently being decoded is the _____.
- 5. Clock speed is measured in _____.
- 6. A processor with four independent processing units on one chip is called a _____ processor.
- 7. _____ usually licenses its processor design to other companies rather than manufacturing chips itself.
- 8. _____ (RISC/CISC) uses a small number of simple instructions, each completing in about one clock cycle.
- 9. _____ (RISC/CISC) uses a larger set of complex instructions, where one instruction can perform several steps.
- 10. Intel and AMD's processors both use the _____ instruction set family.

**Answers**

1. Instruction set (ISA) | 2. Program Counter (PC) | 3. Memory Data Register (MDR) | 4. Instruction Register (IR) | 5. Hertz (Hz) / GHz | 6. Quad-core (multi-core) | 7. ARM | 8. RISC | 9. CISC | 10. x86-64

## 9 True or False

- 1. The instruction set of a CPU can differ between different CPU families.
- 2. The Memory Address Register (MAR) holds the actual data being read from memory.
- 3. Registers are slower than RAM.
- 4. A CPU with a higher clock speed always has more cores.
- 5. ARM usually designs processors but does not manufacture most of the chips itself.
- 6. Intel and AMD both use the x86-64 instruction set.
- 7. RISC processors typically need more instructions to complete the same task compared to CISC.
- 8. Apple's M-series chips are based on the ARM (RISC) architecture.

### Answers with Explanation

**1. True.** Different CPU families, such as x86-64 and ARM, have different instruction sets, which is why software isn't always cross-compatible.

**2. False.** The MAR holds the ADDRESS of the memory location, not the data itself — the data is held by the MDR (Memory Data Register).

**3. False.** Registers are dramatically faster than RAM (over 100 times faster), though there are far fewer of them and each holds far less data.

**4. False.** Clock speed and number of cores are two independent specifications — a chip can have a high clock speed with just one or two cores, or a lower clock speed with many cores.

**5. True.** ARM Holdings mostly licenses its architecture to companies like Qualcomm, Apple, and Samsung, who then manufacture their own chips.

**6. True.** Both Intel and AMD build x86-64 (CISC-style) chips, which is why software compiled for one usually runs on the other.

**7. True.** RISC uses smaller, simpler instructions, so more of them are typically needed to complete the same overall task.

**8. True.** Apple's M-series chips are custom-designed but built on the ARM (RISC) instruction set architecture.

## 10 Short Answer Questions (2–3 Marks)

- What is an instruction set? (2 marks)
- Name any three CPU registers and state what each one holds. (3 marks)
- What is clock speed, and in what unit is it measured? (2 marks)
- Differentiate between a single-core and a multi-core processor. (2 marks)
- Name the three major processor families discussed in this lecture. (2 marks)
- What is the key difference between RISC and CISC? (3 marks)

## 11 Long Answer Model Answers (5, 10 & 15 Marks)

The following are fully written model answers. Use these as a template for structure, depth, and the level of explanation expected in your own exam answer — do not simply memorise them word for word; practise writing them in your own words using the same structure.

### Q1 (5 Marks): What is an instruction set? Explain why different CPUs can have different instruction sets.

An instruction set (also called the Instruction Set Architecture, or ISA) is the complete, fixed vocabulary of every basic command — such as LOAD, STORE, ADD, or COMPARE — that a particular model of CPU is built to understand and carry out. Every program, no matter what language it was originally written in, is ultimately translated down into a sequence of these instructions in machine code before the Control Unit can fetch and execute it.

Different CPU families — such as Intel/AMD's x86-64 chips and ARM-based chips — are built around different instruction sets. This is precisely why an app or program compiled for one type of chip usually cannot run directly on a different type of chip: the receiving CPU simply does not recognise those particular binary patterns as valid instructions, unless the software is translated or emulated specially for it.

### Q2 (10 Marks): Describe the function of the Program Counter (PC), Memory Address Register (MAR), Memory Data Register (MDR), Instruction Register (IR), and Accumulator (AC). Explain how they work together during one Fetch step.

Registers are the CPU's own tiny, extremely fast internal storage locations, each holding only a small amount of data needed for the instruction currently being processed. Five of the most important registers are:

| Register | Function |
|---|---|
| Program Counter (PC) | Holds the memory address of the NEXT instruction to be fetched. |
| Memory Address Register (MAR) | Holds the address of the memory location currently being accessed. |

| Register | Function |
|---|---|
| Memory Data Register (MDR) | Holds the actual data being transferred to or from memory. |
| Instruction Register (IR) | Holds the instruction currently being decoded and executed. |
| Accumulator (AC) | Holds the running result of the ALU's current arithmetic or logical operation. |

These registers cooperate in a fixed sequence during every single Fetch step: the PC's address is copied into the MAR; the CPU reads memory at that address and places the instruction found there into the MDR, which is then copied into the IR for decoding; and finally the PC is incremented so it is ready to point to the following instruction. This repeats, together with Decode and Execute, billions of times per second.

In conclusion, no single register could do this job alone — the PC ensures the CPU always knows what comes next, the MAR and MDR handle the actual traffic to and from memory, the IR holds the current job for the Control Unit to interpret, and the Accumulator captures the ALU's output. Together, they give the CPU exactly the small, fast working memory it needs to run the instruction cycle continuously.

### Q3 (15 Marks): "Two computers may both be described as having a '3.0 GHz processor', yet perform very differently in practice." Discuss this statement with reference to clock speed, number of cores, and the RISC/CISC design of the processor, using suitable examples.

This statement is entirely accurate, and understanding why requires looking beyond the single GHz number on a spec sheet to at least three separate factors: clock speed, core count, and instruction set design.

**Clock speed:** Clock speed, measured in GHz, tells us how many ticks per second a single processor core can perform — a 3.0 GHz processor completes roughly 3 billion ticks every second. However, this only describes the speed of ONE core working through its own steps; it says nothing about how many independent tasks the chip can handle at the same time.

**Number of cores:** A modern CPU may contain several independent cores — dual, quad, octa-core, or more — each capable of executing its own stream of instructions in parallel. A 3.0 GHz processor with 8 cores can genuinely run far more tasks at once than a 3.0 GHz processor with only 2 cores, even though both share the identical "3.0 GHz" clock speed figure.

**RISC vs CISC design:** Beyond raw speed and core count, the instruction set design itself matters. A RISC chip (such as an ARM-based processor) uses simple instructions that typically complete in about one clock cycle, and is generally more power-efficient; a CISC chip (such as an Intel or AMD x86-64 processor) uses more complex instructions that can take several clock cycles but accomplish more per instruction. This is why a RISC-based 3.0 GHz chip and a CISC-based 3.0 GHz chip can produce noticeably different real-world performance and battery life, despite sharing the same clock speed on paper.

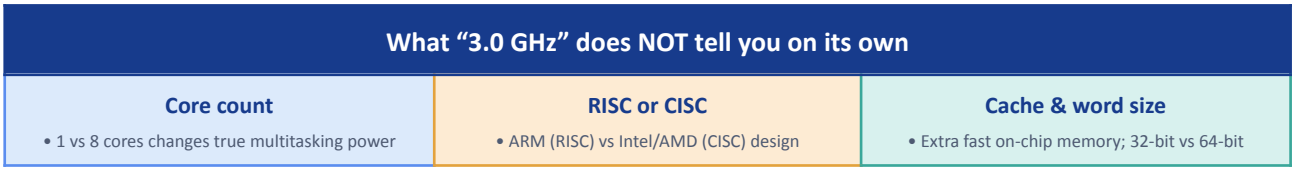


*Fig 6.2 — Clock speed is only one part of the full performance picture.*

In conclusion, the statement is correct: a "3.0 GHz processor" is an incomplete description on its own. Two computers sharing that same clock speed can perform very differently once we account for how many cores each one has and whether the underlying architecture follows the RISC or CISC design philosophy — exactly as seen when comparing a modern ARM-based laptop against a traditional Intel or AMD x86-64 laptop.

## 12 Exam Tips

- Memory trick for registers: “MAR before it goes (the address), MDR when it arrives (the data), PC always points forward, IR holds what's now.”
- Always state the unit for clock speed (GHz), and never confuse it with the number of cores — they are two separate specifications.
- For “RISC vs CISC” questions, always name at least one real example chip family for each side: ARM for RISC, Intel/AMD for CISC.
- Remember: ARM is mainly a design/architecture licensor, not always a chip manufacturer itself — this distinction is a favourite trick question.
- For 10 and 15 mark questions, structure your answer with a clear paragraph (or table) for each component, exactly like the model answers in Section 11 — examiners reward clear structure.
- Common Mistake: Do not say “ARM makes phones” — say “ARM designs the architecture used in most phone processors, which other companies then manufacture.”
- Common Mistake: Do not treat clock speed as the only measure of a processor's performance — always mention core count and RISC/CISC design too.

---

*Lecture 6 — Instruction Set, Registers, Processor Speed and Types of Processors — continues directly from Lecture 5's exploration of the CPU.*